AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Drzewo BTS

Autor:
Jelito Tomasz
Basta Jakub
Basta Damian

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań 4

1.1. Cele i założenia projektu 5

1.2. Środowisko technologiczne 10

2. Analiza problemu 11

2.1. Zastosowanie struktury Drzewa Poszukiwań Binarnych w informatyce . . 11

2.2. Szczegółowy sposób działania algorytmów na drzewie BST 12

2.2.1. Wstawianie elementu (Insert) 12

2.2.2. Wyszukiwanie i wyznaczanie ścieżki (FindPath) 12

2.2.3. Usuwanie elementu (Remove) 13

2.2.4. Algorytmy przejścia przez drzewo (Traversals) 13

2.2.5. Serializacja i deserializacja danych (FileManager) 13

2.3. Szczegółowy przykład wykonania algorytmu krok po kroku 14

2.4. Analiza narzędzi inżynierii oprogramowania: Git oraz Doxygen 16

2.4.1. System kontroli wersji Git 16

2.4.2. Narzędzie Doxygen 17

3. Projektowanie 18

3.1. Środowisko programistyczne i wykorzystane narzędzia 18

3.2. Architektura systemu i diagram klas UML 19

3.3. Schematy działania głównych algorytmów 21

3.3.1. Algorytm wstawiania elementu (Insert) 21

3.3.2. Algorytm wyznaczania ścieżki (FindPath) 21

3.3.3. Algorytm binarnej serializacji i deserializacji 21

3.4. Metodyka pracy z narzędziem Git i zarządzanie konfliktami 22

3.4.1. Strategia tworzenia gałęzi i przebieg pracy 22

3.4.2. Procedura wywoływania i rozwiązywania konfliktów 23

3.5. Przebieg prac w systemie kontroli wersji Git 23

3.5.1. Etap 1: Inicjalizacja repozytorium i praca sekwencyjna 23

3.5.2. Etap 2: Równoległa praca na gałęziach i rozwiązywanie 6 konfliktów 28

4. Implementacja **36**

4.1. Implementacja klas programu i listingi kodu 36

4.1.1. Węzeł drzewa – Node.h 36

4.1.2. Klasa Drzewa BST – BST.h i BST.cpp 36

4.1.3. Obsługa plików – FileManager.h i FileManager.cpp 39

4.1.4. Główny moduł aplikacji – main.cpp 40

4.2. Opis wyników działania programu 41

5. Wnioski **42**

5.1. Wnioski dotyczące implementacji algorytmów i struktury BST 42

5.2. Wnioski dotyczące pracy zespołowej i systemu kontroli wersji Git 43

5.3. Podsumowanie końcowe 43

Literatura **45**

Spis rysunków **46**

Spis tabel **47**

Spis listingów **48**

1. Ogólne określenie wymagań

Niniejsza dokumentacja dotyczy projektu realizowanego w ramach przedmiotu Programowanie zaawansowane - P2 w Akademii Nauk Stosowanych w Nowym Sączu. Głównym celem pracy jest zaprojektowanie, zaimplementowanie oraz przetestowanie struktury danych znanej jako Drzewo Poszukiwań Binarnych (ang. *Binary Search Tree* - BST) przy wykorzystaniu języka C++ oraz kontenerów biblioteki standardowej STL.

Projekt ten jest realizowany w grupach dwuosobowych, co ma na celu nie tylko sprawdzenie umiejętności programistycznych, ale również rozwinięcie kompetencji miękkich oraz technicznych związanych z pracą zespołową z wykorzystaniem rozproszonych systemów kontroli wersji. Zgodnie z rygorystycznymi wytycznymi, cały proces twórczy musi odbywać się bez wsparcia jakichkolwiek narzędzi opartych na sztucznej inteligencji, takich jak ChatGPT czy GitHub Copilot, co ma zagwarantować samodzielność i pełne zrozumienie implementowanych algorytmów.

Zbudowana aplikacja musi opierać się na paradygmacie programowania obiektowego. Oznacza to, że logika programu ma zostać hermetyzowana w dedykowanych klasach. Pierwsza z klas ma odpowiadać wyłącznie za strukturę i logikę operacji na drzewie BST. Klasa ta musi dostarczać interfejs publiczny (metody) umożliwiający:

- dodanie nowego elementu do struktury,
- usunięcie wskazanego elementu,
- całkowite wyczyszczenie drzewa (usunięcie wszystkich węzłów z pamięci),
- wyszukiwanie ścieżki prowadzącej od korzenia do podanego przez użytkownika elementu.

Kolejnym istotnym wymaganiem jest możliwość graficznego wyświetlenia drzewa na ekranie konsoli, przy czym użytkownik podczas działania programu musi mieć możliwość wyboru jednej z trzech klasycznych metod przechodzenia przez drzewo: preorder, inorder oraz postorder. Aplikacja musi implementować wszystkie te trzy algorytmy.

Kolejnym komponentem systemu, oddzielonym architektonicznie od samej struktury drzewa, jest klasa zarządzająca operacjami wejścia/wyjścia (I/O). Jej zadaniem jest trwała serializacja i deserializacja danych. Wymagane jest zaimplementowanie metod pozwalających na zapis utworzonego drzewa BST do

pliku binarnego oraz odczyt z takiego pliku w celu odtworzenia pełnej struktury w pamięci RAM.

Dodatkowo program musi posiadać funkcjonalność czytania danych z płaskiego pliku tekstowego (zawierającego ciąg liczb), co pozwoli na masowe wprowadzanie danych i budowanie drzewa od podstaw lub rozbudowywanie już istniejącego, pustego lub częściowo wypełnionego drzewa. Możliwy musi być również zapis drzewa wygenerowanego w programie do pliku tekstowego.

Całość funkcjonalności ma być spięta z poziomu funkcji `main`, w której zaimplementowane zostanie interaktywne menu dla użytkownika. Menu to ma wyświetlać opcje zarządzania drzewem oraz plikami, oczekiwać na decyzję i poprawnie reagować na wybór. Istotnym aspektem jest zapewnienie odpowiedniej instrukcji wyjścia z programu, pozwalającej na poprawne zamknięcie aplikacji i zwolnienie zasobów pamięciowych. Architektura plików projektu wymusza, aby wszystkie klasy oraz funkcja `main` znajdowały się w odrębnych plikach źródłowych i nagłówkowych.

Aspekt zarządzania kodem i pracy zespołowej opiera się na platformie GitHub. Każdy z członków dwuosobowego zespołu zobligowany jest do aktywnego udziału w tworzeniu repozytorium, co musi zostać udokumentowane poprzez wykonanie co najmniej 5 commitów przez każdą z osób. Praca ma symulować rzeczywiste warunki komercyjne - wymaga się tworzenia osobnych gałęzi (ang. *branch*) dla implementowanych funkcjonalności, a następnie ich scalania (ang. *merge*) do gałęzi głównej.

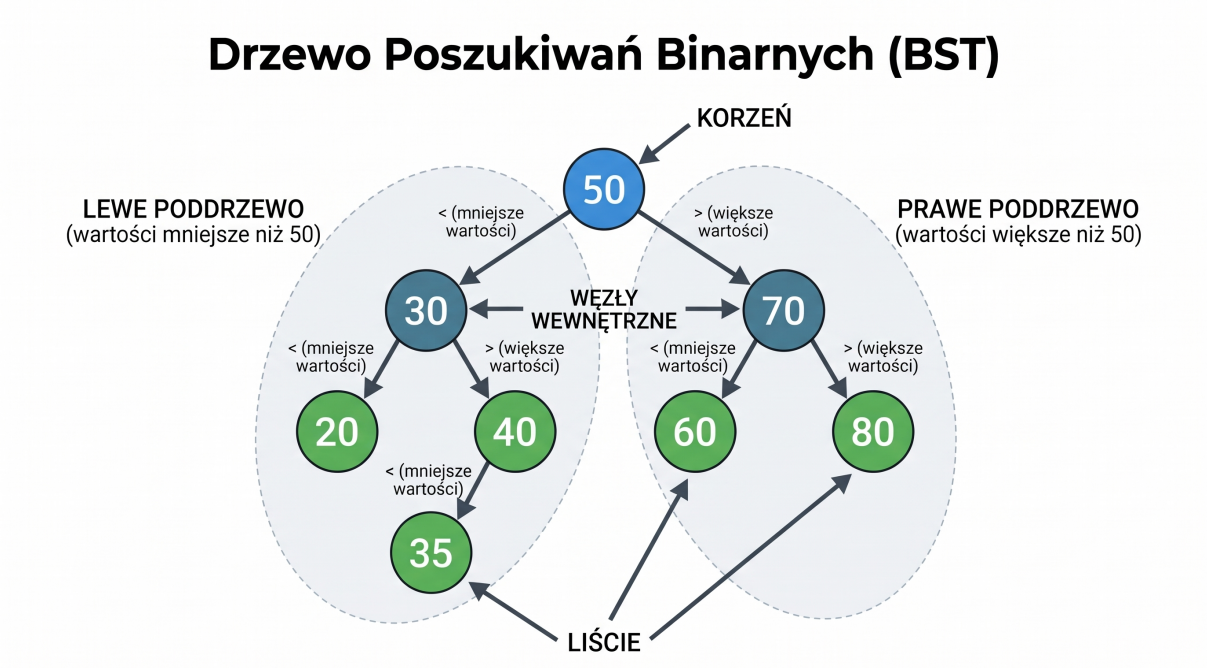
Specyfikacja wymaga zademonstrowania pracy sekwencyjnej (jedna osoba tworzy gałąź i ją scala, potem robi to druga) oraz równoległej (obie osoby pracują na swoich gałęziach w tym samym czasie). Kluczowym zadaniem jest celowe doprowadzenie do co najmniej 6 konfliktów podczas scalania kodu i ich poprawne rozwiązanie (po 3 na każdego członka zespołu). Wyniki tych operacji muszą zostać uwidocznione w dokumentacji w postaci zrzutów ekranu (tzw. drzewo commitów z GitHuba). Dokumentacja kodu wewnątrz plików ma zostać wygenerowana dodatkowo za pomocą narzędzia Doxygen.

1.1. Cele i założenia projektu

Aby w pełni zrozumieć istotę zaimplementowanego programu, konieczne jest omówienie teoretycznych podstaw Drzewa Poszukiwań Binarnych (BST) oraz zobrazowanie go odpowiednim przykładem graficznym. Struktura drzewiasta jest jedną z fundamentalnych struktur danych w informatyce, stanowiącą rozszerzenie

koncepcji listy jednokierunkowej czy dwukierunkowej. O ile lista składa się z elementów (węzłów) ułożonych liniowo, gdzie każdy element wskazuje na jeden element następny, o tyle w drzewie binarnym element może mieć wskaźniki na maksymalnie dwa kolejne węzły (tzw. lewego i prawego potomka).

Drzewo BST charakteryzuje się specyficznym ułożeniem danych, które gwarantuje utrzymanie porządku sortowania w czasie rzeczywistym. Główna właściwość drzewa BST mówi, że dla dowolnego węzła X, wszystkie wartości znajdujące się w jego lewym poddrzewie muszą być mniejsze od wartości przechowywanej w X, natomiast wszystkie wartości w prawym poddrzewie muszą być od niej większe. Ta prosta własność pozwala na osiągnięcie logarytmicznej złożoności czasowej $O(\log n)$ dla operacji wyszukiwania, dodawania i usuwania elementów w optymalnym przypadku, co daje drzewom BST ogromną przewagę wydajnościową nad klasycznymi tablicami czy listami w przypadku dużych zbiorów danych[1].



Rys. 1.1. Graficzna reprezentacja struktury Drzewa Poszukiwań Binarnych z widocznymi relacjami węzłów. Zdjęcie wygenerowany przy pomocy AI.

Rysunek 2.2 (s. 16) obrazuje schemat ideowy drzewa BST. Zaczyna się ono od elementu najwyższego, zwanego **korzeniem** (ang. *root*). Jeśli korzeń przechowuje

[1]Szczegółowy opis własności algorytmicznych drzew BST znajduje się w literaturze przedmiotu [1].

wartość np. 50, to wszystkie elementy w lewej części grafu będą mniejsze od 50, a w prawej większe. Węzły, które nie posiadają żadnych potomków (ich wskaźniki na lewe i prawe dziecko mają wartość `nullptr`), nazywane są **liśćmi**.

Przykład operacji wstawiania (Insert): Wyobraźmy sobie pustą strukturę. Użytkownik w programie wybiera opcję dodania elementu o wartości 40. Drzewo jest puste, więc 40 staje się korzeniem. Następnie użytkownik dodaje 20. Algorytm zaczyna od korzenia (40). Ponieważ 20 jest mniejsze od 40, algorytm przechodzi w lewo. Miejsce to jest puste, więc tworzony jest nowy węzeł z wartością 20. Następnie użytkownik dodaje wartość 60. Algorytm znów startuje od korzenia (40), stwierdza, że $60 > 40$, przechodzi w prawo i tworzy węzeł. Kiedy po tym użytkownik spróbuje dodać wartość 30, algorytm przejdzie ścieżką: od korzenia (40) w lewo (bo $30 < 40$), następnie porówna 30 z węzłem 20. Ponieważ $30 > 20$, przejdzie w prawo i utworzy węzeł jako prawe dziecko dwudziestki.

Przykład operacji usuwania (Remove): Operacja usuwania jest procesem znacznie bardziej skomplikowanym niż wstawianie i musi obsłużyć trzy różne przypadki:

1. **Usuwanie liścia:** Algorytm po prostu zwalnia pamięć, a u rodzica ustawia wskaźnik na `nullptr`.

2. **Usuwanie węzła z jednym dzieckiem:** Dziecko zajmuje miejsce usuwanego węzła, podłączając się bezpośrednio do jego rodzica.

3. **Usuwanie węzła z dwojgiem dzieci:** Węzeł ten zastępuje się zazwyczaj najmniejszym elementem z jego prawego poddrzewa (lub największym z lewego poddrzewa), po czym fizycznie usuwa ów sklonowany węzeł zastępczy z jego oryginalnej pozycji.

Metody przechodzenia przez drzewo: W projekcie wymogiem było zaimplementowanie trzech algorytmów przechodzenia przez drzewo (odwiedzania jego węzłów). Algorytmy te mają charakter rekurencyjny:

1. **Pre-order (korzeń, lewy, prawy):** najpierw przetwarzana jest wartość węzła bieżącego, następnie algorytm wywołuje się rekurencyjnie dla całego lewego poddrzewa, a na końcu dla prawego. Ta metoda bywa wykorzystywana m.in. do tworzenia dokładnej kopii struktury drzewa.

2. **In-order (lewy, korzeń, prawy):** to metoda, która odwiedza najpierw lewe poddrzewo, wypisuje wartość bieżącą i przechodzi do prawego poddrzewa.

Właściwość drzewa BST sprawia, że odczyt drzewa metodą In-order zwraca ciąg liczb posortowanych rosnąco.

3. **Post-order (lewy, prawy, korzeń):** najpierw algorytm zagłębia się we wszystkie poddrzewa (lewe, potem prawe), a sam węzeł główny poddawany jest operacji na samym końcu. Ten sposób przejścia jest idealny do całkowitego usuwania drzewa z pamięci (jak to zrealizowano w metodzie `clear`), ponieważ gwarantuje, że nie usuniemy węzła-rodzica zanim bezpiecznie nie usuniemy jego dzieci.

Ze względu na wymagania projektowe, w dalszej części pracy implementacja wyżej wymienionych algorytmów zostanie zaprezentowana w formie listingów kodu.

Tab. 1.1. Złożoność obliczeniowa podstawowych operacji drzewa BST

Operacja	Przypadek średni	Przypadek najgorszy (drzewo zdegenerowane)
Wyszukiwanie	$O(\log n)$	$O(n)$
Wstawianie	$O(\log n)$	$O(n)$
Usuwanie	$O(\log n)$	$O(n)$

Tabela 1.1 (s. 8) obrazuje średnią i najgorszą złożoność czasową. Należy zaznaczyć, że dla poprawnego zaimplementowania logiki, w tym serializacji do plików, istotne jest również zrozumienie wejścia/wyjścia systemu. Jak widać w listingu 1 (s. 8), praca na danych binarnych (metoda `saveBinary`) różni się od tekstowej, gdyż pozwala na zrzut surowej formy pamięci bezpośrednio na nośnik fizyczny dysku, co minimalizuje czas potrzebny na wczytanie bardzo dużych struktur danych w przyszłości, lecz uniemożliwia bezpośrednie czytanie takiego pliku przez człowieka.

```
#include <iostream>
#include <fstream>
using namespace std;

// Zarys logiczny mechanizmu binarnego
void ZapiszWymaganyPlik(string nazwa) {
    ofstream out(nazwa, ios::binary);
    // tutaj przebiega rekurencyjny zrzut pamieci
    // zapobiegajacy problemom ze znacznikami konca pliku
    out.close();
}
```

Listing 1. Przykładowy zarys serializacji (pseudo-struktura)

Listing 1 przedstawia wyłącznie poglądową konstrukcję mechanizmu zarządzania plikiem ze strumieniem wyjściowym typu binarnego. Rzeczywisty kod projektu z metodami klasy `FileManager` znajduje się w dziale Implementacja.

1.2. Środowisko technologiczne

Realizacja tak złożonego projektu grupowego wymagała przygotowania ujednoliconego środowiska pracy. Poniżej są opisane kroki potrzebne do instalacji narzędzi, systemu kontroli wersji Git, kompilatorów, a także systemu składu tekstu LaTeX, niezbędnego do przygotowania niniejszego szablonu dokumentacji.

Kompilator C++ i edytor: Podstawą pracy z kodem źródłowym był edytor Visual Studio Code ze względu na polecaną przez zadanie wtyczkę GitHub. Aby kod w C++ z wykorzystaniem biblioteki STL kompilował się poprawnie, zainstalowano zestaw kompilatorów MinGW (wersja dla systemu Windows) obsługujących standard C++11/C++14 lub wyższy. Zmienne środowiskowe (`PATH`) systemu zostały odpowiednio zaktualizowane, by umożliwić dostęp do polecenia `g++` z poziomu terminala.

System kontroli wersji Git: Każdy ze współtwórców pobrał i zainstalował klienta Git for Windows. Po instalacji wymagana była wstępna konfiguracja globalnych danych użytkownika przy pomocy komend w terminalu: `git config --global user.name` oraz `git config --global user.email`. Umożliwiło to przypisanie poprawnych sygnatur autorskich do każdego z 5 wymaganych commitów. W samym Visual Studio Code doinstalowano oficjalne rozszerzenia GitHub Pull Requests and Issues oraz GitLens, które ułatwiły późniejsze symulowanie pracy równoległej na odrębnych gałęziach (branches) oraz zautomatyzowały wizualne wsparcie przy rozwiązywaniu celowo tworzonych konfliktów przy operacjach `merge`.

Dokumentacja: Zgodnie z wytycznymi, do automatycznego generowania dokumentacji z kodu źródłowego wykorzystano oprogramowanie Doxygen. Wymagało to dołączenia specjalnych bloków komentarzy do klas i metod w plikach nagłówkowych `.h`.

2. Analiza problemu

Rozdział ten poświęcony jest szczegółowej analizie teoretycznej oraz praktycznej problematyki związanej z strukturami drzewiastymi, ze szczególnym uwzględnieniem Drzew Poszukiwań Binarnych (BST). Przeanalizowano w nim dziedziny informatyki, w których algorytmy drzewiaste odgrywają kluczową rolę, opisano ze szczegółami mechanizmy działania wszystkich operacji na drzewie BST, przedstawiono krok po kroku przykładowy przebieg działania algorytmu oraz opisano narzędzia inżynierii oprogramowania (Git, Doxygen) użyte w projekcie.

2.1. Zastosowanie struktury Drzewa Poszukiwań Binarnych w informatyce

Drzewa poszukiwań binarnych oraz ich zaawansowane modyfikacje stanowią jeden z fundamentów współczesnej informatyki. Dzięki zdolności do utrzymywania posortowanego zbioru danych przy jednoczesnym zapewnieniu dynamicznego wstawiania i usuwania elementów, znajdują one zastosowanie w wielu kluczowych obszarach systemów komputerowych:

- **Systemy baz danych i indeksowanie:** Choć w wielkich relacyjnych bazach danych stosuje się warianty wielodrożne (np. B-Drzewa lub B+ Drzewa), to podstawowa koncepcja podziału przestrzeni kluczy i szybkiego nawigowania po węzłach pochodzi bezpośrednio z drzew BST. Indeksy w bazach danych wykorzystują drzewiaste struktury, aby zredukować liczbę operacji dyskowych I/O z $O(n)$ do $O(\log n)$.
- **Systemy plików:** Nowoczesne systemy plików (takie jak NTFS, EXT4 czy Btrfs) wykorzystują drzewiaste struktury danych do organizacji katalogów, zarządzania alokacją bloków na dysku oraz szybkiego wyszukiwania plików po nazwie lub identyfikatorze inode.
- **Kompilatory i parsowanie kodu (Abstract Syntax Trees - AST):** W procesie kompilacji programu kod źródłowy jest przekształcany przez analizator składniowy w drzewo składniowe (AST). Reprezentuje ono strukturę gramatyczną programu i umożliwia kompilatorowi wygenerowanie optymalnego kodu maszynowego.
- **Tablice asocjacyjne i kontenery STL:** W języku C++ kontener `std::set` oraz `std::map` najczęściej zaimplementowane są w oparciu o zbalansowaną

odmianę drzewa BST (drzewo czerwono-czarne). Zapewnia to przewidywalny czas wykonania operacji wyszukiwania, dodawania i usuwania wynoszący $O(\log n)$.

- **Algorytmy kompresji danych:** Algorytm Huffmana, stosowany m.in. w formatach ZIP, JPEG czy MP3, opiera się na budowaniu binarnego drzewa kodowego, gdzie częściej występujące symbole mają krótsze ścieżki od korzenia.

2.2. Szczegółowy sposób działania algorytmów na drzewie BST

Struktura BST opiera się na węzłach zawierających wartość (`data`) oraz dwa wskaźniki: lewe dziecko (`left`) i prawe dziecko (`right`). Istotą działania algorytmów jest bezwzględne utrzymanie niezmiennika BST: dla każdego węzła V, wszystkie wartości w lewym poddrzewie V są mniejsze od V.data, a w prawym poddrzewie większe od V.data.

2.2.1. Wstawianie elementu (Insert)

Algorytm wstawiania nowego elementu o wartości K przebiega rekurencyjnie:

1. Jeśli drzewo jest puste (wskaźnik na poddrzewo wynosi `nullptr`), tworzony jest nowy węzeł o wartości K i zwracany jako korzeń danego poddrzewa.
2. Jeśli $K <$ node $\rightarrow$ data, algorytm przechodzi rekurencyjnie do lewego potomka: node $\rightarrow$ left = insert(node $\rightarrow$ left, K).
3. Jeśli $K >$ node $\rightarrow$ data, algorytm przechodzi rekurencyjnie do prawego potomka: node $\rightarrow$ right = insert(node $\rightarrow$ right, K).
4. Jeśli $K ==$ node $\rightarrow$ data, wartość już istnieje w drzewie i jest ignorowana (brak dublikatów).

2.2.2. Wyszukiwanie i wyznaczanie ścieżki (FindPath)

Wyszukiwanie drogi do elementu polega na przechodzeniu od korzenia w dół drzewa i rejestrowaniu odwiedzanych węzłów w kontenerze `std::vector<int>`. Rekurencyjna funkcja dopisuje bieżący węzeł do wektora ścieżki. Jeśli odnawiany węzeł zawiera poszukiwaną wartość, zwracana jest prawda (`true`). W przeciwnym

razie algorytm próbuje znaleźć wartość w lewym lub prawym poddrzewie. Jeśli żadna ścieżka nie doprowadzi do szukanej wartości, ostani element jest usuwany z wektora (`pop_back()`), a funkcja zwraca fałsz (`false`).

2.2.3. Usuwanie elementu (Remove)

Usuwanie elementu z drzewa BST wymaga obsłużenia trzech odrębnych przypadków strukturalnych:

- **Przypadek 1: Węzeł jest liściem (brak dzieci).** Pamięć zajmowana przez węzeł zostaje zwolniona komendą `delete`, a wskaźnik rodzica zostaje ustawiony na `nullptr`.
- **Przypadek 2: Węzeł posiada jedno dziecko (lewe lub prawe).** Węzeł zostaje usunięty z pamięci, a jego jedyne dziecko zajmuje jego miejsce, łącząc się bezpośrednio z rodzicem usuwanego węzła.
- **Przypadek 3: Węzeł posiada dwoje dzieci.** Nie można bezpośrednio usunąć węzła bez naruszenia spójności drzewa. Algorytm znajduje tzw. **następnika** (ang. *successor*), czyli węzeł o minimalnej wartości w prawym poddrzewie usuwanego elementu (funkcja `findMin`). Wartość następnika jest kopiowana do usuwanego węzła, po czym algorytm wywołuje rekurencyjnie usuwanie następnika z prawego poddrzewa (co sprowadza się już do Przypadku 1 lub 2).

2.2.4. Algorytmy przejścia przez drzewo (Traversals)

Przechodzenie przez strukturę drzewa polega na odwiedzeniu każdego węzła dokładnie jeden raz. W zależności od kolejności przetwarzania węzła głównego w stosunku do jego dzieci wyróżnia się trzy metody:

- **Preorder (N-L-R):** Węzeł → Lewe poddrzewo → Prawe poddrzewo.
- **Inorder (L-N-R):** Lewe poddrzewo → Węzeł → Prawe poddrzewo. Zwraca elementy w kolejności posortowanej rosnąco.
- **Postorder (L-R-N):** Lewe poddrzewo → Prawe poddrzewo → Węzeł. Stosowane m.in. przy destrukcji drzewa i zwalnianiu pamięci.

2.2.5. Serializacja i deserializacja danych (FileManager)

Trwałe zapisywanie struktury drzewiastej na dysku wymaga zachowania informacji nie tylko o wartościach, ale również o kształcie drzewa. Wklasycznej serializacji

binarnej (`FileManager::serialize`) stosuje się zapis w porządku pre-order, gdzie brakujące poddrzewa (`nullptr`) zapisywane są ze specjalną wartością kontrolną (tzw. markerem pustym, np. -999999). Podczas deserializacji binarnej algorytm odtwarza tożsamo odnośniki wskaźnikowe.

2.3. Szczegółowy przykład wykonania algorytmu krok po kroku

W celu dokładnego zaprezentowania sposobu funkcjonowania drzewa BST przeanalizujmy wykonanie serii operacji na konkretnym przykładzie liczbowym.

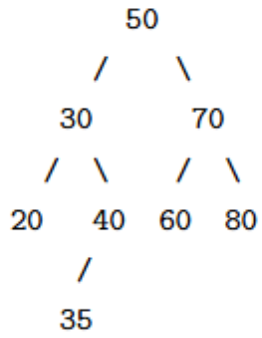
Krok 1: Budowanie drzewa. Do pustego drzewa wstawiamy kolejno następujący ciąg elementów: $[50, 30, 70, 20, 40, 60, 80, 35]$.

1. Wstawienie 50: Drzewo było puste. 50 staje się korzeniem.
2. Wstawienie 30: $30 < 50 \rightarrow$ trafia jako lewe dziecko węzła 50.
3. Wstawienie 70: $70 > 50 \rightarrow$ trafia jako prawe dziecko węzła 50.
4. Wstawienie 20: $20 < 50 \rightarrow$ lewo do 30; $20 < 30 \rightarrow$ trafia jako lewe dziecko 30.
5. Wstawienie 40: $40 < 50 \rightarrow$ lewo do 30; $40 > 30 \rightarrow$ trafia jako prawe dziecko 30.
6. Wstawienie 60: $60 > 50 \rightarrow$ prawo do 70; $60 < 70 \rightarrow$ trafia jako lewe dziecko 70.
7. Wstawienie 80: $80 > 50 \rightarrow$ prawo do 70; $80 > 70 \rightarrow$ trafia jako prawe dziecko 70.
8. Wstawienie 35: $35 < 50 \rightarrow$ lewo do 30; $35 > 30 \rightarrow$ prawo do 40; $35 < 40 \rightarrow$ trafia jako lewe dziecko 40.

   Po wykonaniu powyzszych operacji struktura drzewa przybiera następującą postać logiczną:

Krok 2: Odczyt w trzech metodach przechodzenia.

- **Preorder (N-L-R):** $50 \rightarrow 30 \rightarrow 20 \rightarrow 40 \rightarrow 35 \rightarrow 70 \rightarrow 60 \rightarrow 80$
- **Inorder (L-N-R):** $20 \rightarrow 30 \rightarrow 35 \rightarrow 40 \rightarrow 50 \rightarrow 60 \rightarrow 70 \rightarrow 80$ (ciąg posortowany rosnąco)
- **Postorder (L-R-N):** $20 \rightarrow 35 \rightarrow 40 \rightarrow 30 \rightarrow 60 \rightarrow 80 \rightarrow 70 \rightarrow 50$



Rys. 2.1. Przedstawiono postać logiczną.

Krok 3: Wyszukiwanie drogi do elementu 35 **(findPath).**

(a) Start w korzeniu: dodaj 50 do wektora ścieżki. $35 \neq 50$, $35 < 50 \rightarrow$ przejście w lewo.

(b) Odwiedź 30: dodaj 30 do wektora ścieżki. $35 \neq 30$, $35 > 30 \rightarrow$ przejście w prawo.

(c) Odwiedź 40: dodaj 40 do wektora ścieżki. $35 \neq 40$, $35 < 40 \rightarrow$ przejście w lewo.

(d) Odwiedź 35: dodaj 35 do wektora ścieżki. Znaleziono element!

(e) **Wynik wyjściowy:** Droga: $50 \rightarrow 30 \rightarrow 40 \rightarrow 35$.

Krok 4: Usuwanie elementu 30 **(Przypadek 3 - dwoje dzieci).**

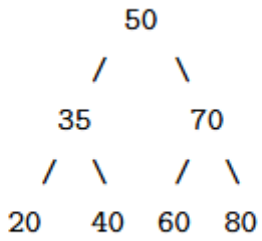
(a) Węzeł 30 posiada lewe dziecko (20) oraz prawe poddrzewo z ukorzenieniem w 40.

(b) Algorytm szuka minimum w prawym poddrzewie trójki (węzeł 40): przechodzi w lewo z 40 i znajduje węzeł 35.

(c) Wartość 35 jest podmieniana wmiejsce usuwanego węzła 30.

(d) Następnie usuwany jest oryginalny węzeł 35 z prawego poddrzewa (przypadek liścia).

Drzewo po usunięciu elementu 30 wygląda następująco:



Rys. 2.2. Przedstawienie drzewa po usunięciu 30 elementów.

2.4. Analiza narzędzi inżynierii oprogramowania: Git oraz Doxygen

Realizacja projektu w zespole dwuosobowym wymagała wykorzystania profesjonalnego oprogramowania wspomagającego rozwój oprogramowania i automatyzującego dokumentację.

2.4.1. System kontroli wersji Git

Git jest rozproszonym systemem kontroli wersji, który śledzi historie zmian w plikach źródłowych. Historia projektu przechowywana jest w postaci skierowanego grafu acyklicznego (DAG - *Directed Acyclic Graph*) commitów.

- **Gałęzie (Branches):** Gałąź w systemie Git jest lekkim, modyfikowalnym wskaźnikiem na konkretny commit. Tworzenie gałęzi (`git branch <nazwa>`) pozwala programistom rozwijać nowe funkcjonalności w izolacji od stabilnej wersji projektu znajdującej się na gałęzi `main`.
- **Scalanie gałęzi (Merge):** Operacja scalania polega na połączeniu historii dwóch gałęzi. Wyróżnia się dwa główne typy scalania:
  (a) *Fast-forward merge:* Następuje w sytuacji, gdy od momentu utworzenia gałęzi pomocniczej na gałęzi głównej nie wykonano żadnych nowych commitów. Wskaźnik gałęzi głównej jest po prostu przesuwany do przodu.
  (b) *3-way merge:* Następuje, gdy obie gałęzie rozeszły się i posiadają własne, niezależne commity. Git tworzy specjalny commit scalający (*merge commit*), biorąc pod uwagę przodka wspólnego oraz dwa ostatnie commity obu gałęzi.

- **Konflikty scalania (Merge Conflicts):** Konflikt występuje wtedy, gdy dwóch programistów zmodyfikuje te same linie kodu w tym samym pliku na odrębnych gałęziach. Git nie jest wówczas w stanie automatycznie rozstrzygnąć, która wersja jest poprawna, i przerywa proces scalania, wstawiając do pliku markery konfliktów:

```
<<<<<<< HEAD
    void insert(int data); // Zmiana z galezi glownej
=======
    void insertNode(int value); // Zmiana z galezi bocznej
>>>>>>> feature/bst-rename
```

  Programista musi ręcznie przeanalizować kod, wybrać właściwą wersję, usunąć markery i zatwierdzić rozwiązany konflikt nowym commitem.

2.4.2. Narzędzie Doxygen

Doxygen to standardowe narzędzie używane do automatycznego generowania dokumentacji technicznej bezpośrednio z komentarzy umieszczonych w kodzie źródłowym C++. Narzędzie to analizuje nagłówki plików i tworzy czytelną dokumentację w formatach HTML oraz LaTeX. Komentarze przeznaczone dla Doxygena rozpoczynają się od bloku `/**` lub `///` i wykorzystują specjalne tagi, takie jak `@brief` (krótki opis), `@param` (opis parametru funkcji) oraz `@return` (opis wartości zwracanej).

3. Projektowanie

Niniejszy rozdział przedstawia etap projektowy aplikacji realizującej strukturę Drzewa Poszukiwań Binarnych (BST). Szczegółowo omówiono w nim stos technologiczny, konfigurację kompilatora, architekturę obiektową systemu zaprezentowaną za pomocą diagramu klas UML, schematy blokowe kluczowych algorytmów oraz pełną procedurę i strategię wykorzystania narzędzia kontroli wersji Git podczas pracy zespołowej.

3.1. Środowisko programistyczne i wykorzystane narzędzia

W celu realizacji projektu dobrano narzędzia gwarantujące wysoką wydajność, przenieśność oraz zgodność z najnowszymi standardami programowania obiektowego w języku C++.

- **Język programowania:** C++ (zgodny ze standardem C++11/C++14). Wybór ten podyktowany był koniecznością zarządzania wskaźnikami, dynamicznym alokowaniem pamięci (operatory `new` i `delete`) oraz wykorzystaniem kontenerów Standard Template Library (STL), takich jak `std::vector` czy `std::string`.
- **Kompilator i flaga kompilacji:** Zestaw kompilatorów GCC/G++ z dystrybucji MinGW-w64. Kompilacja projektu realizowana jest z wykorzystaniem następujących flag kompilatora:
  - `-std=c++11` – wymuszenie obsługi standardu C++11 (np. stała `nullptr`),
  - `-Wall -Wextra` – włączenie wszystkich ostrzeżeń kompilatora w celu wyeliminowania wycieków pamięci i niebezpiecznych konwersji,
  - `-O2` – optymalizacja kodu wynikowego pod kątem szybkości wykonywania.
- **Zintegrowane Środowisko Programistyczne (IDE):** Visual Studio Code. Środowisko to zapewniło obsługę kodu C++ oraz pełną integrację z wtyczkami systemu kontroli wersji Git (wtyczka *GitHub Pull Requests and Issues* oraz *GitLens*).
- **System Kontroli Wersji:** Git oraz platforma GitHub do prowadzenia zdalnego repozytorium projektu.

- **Generator Dokumentacji:** Doxygen do automatycznego generowania dokumentacji technicznej z komentarzy nagłówkowych.

3.2. Architektura systemu i diagram klas UML

Projekt został zaprojektowany zgodnie z zasadą pojedynczej odpowiedzialności (ang. *Single Responsibility Principle*). Wyodrębniono trzy główne komponenty logiczne ujęte w osobnych plikach nagłówkowych (`.h`) i źródłowych (`.cpp`):

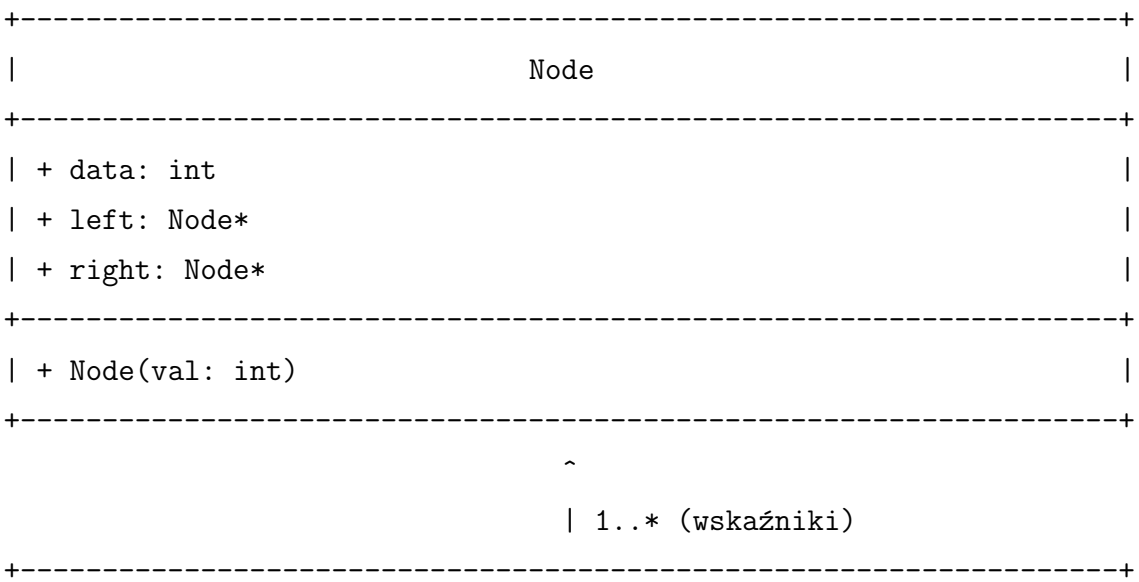
(a) **Struktura Node (`Node.h`):** Odpowiada za reprezentację pojedynczego węzła drzewa. Przechowuje wartość typu `int` oraz dwa wskaźniki na potomków (`left` i `right`).

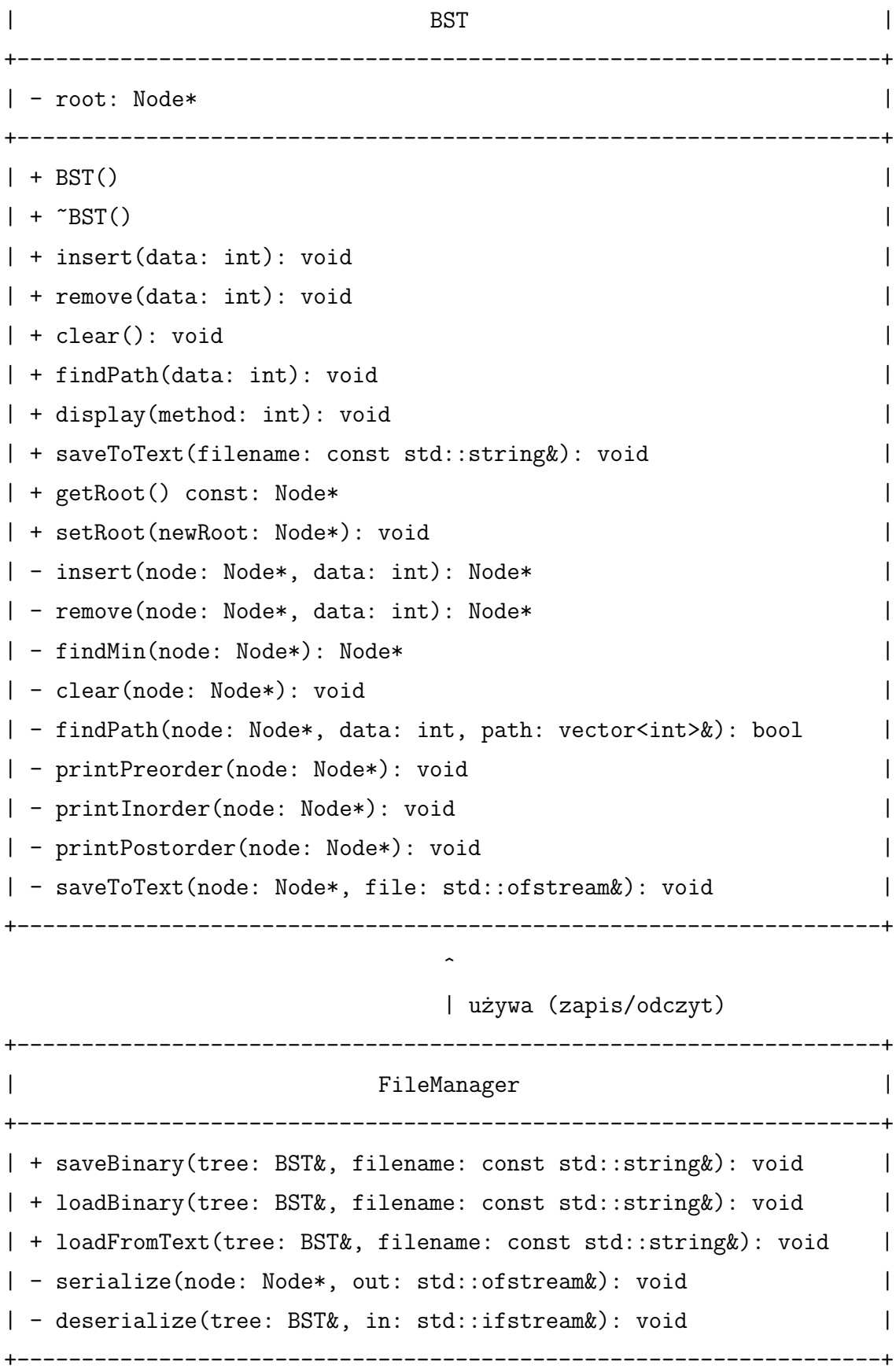
(b) **Klasa BST (`BST.h`, `BST.cpp`):** Hermetyzuje całą strukturę drzewa oraz algorytmy wyszukiwania, dodawania, usuwania i przejść po drzewie (preorder, inorder, postorder).

(c) **Klasa FileManager (`FileManager.h`, `FileManager.cpp`):** Odpowiada za operacje wejścia/wyjścia – czytanie danych z pliku tekstowego oraz binarną serializację/deserializację drzewa.

(d) **Moduł główny (`main.cpp`):** Realizuje interfejs użytkownika w postaci konsolowego menu i steruje przebiegiem aplikacji.

Poniższy diagram w konwencji UML przedstawia relacje i architekturę obiektową zaimplementowanych klas:




BST
- root: Node*
+ BST()
+ ~BST()
+ insert(data: int): void
+ remove(data: int): void
+ clear(): void
+ findPath(data: int): void
+ display(method: int): void
+ saveToText(filename: const std::string&): void
+ getRoot() const: Node*
+ setRoot(newRoot: Node*): void
- insert(node: Node*, data: int): Node*
- remove(node: Node*, data: int): Node*
- findMin(node: Node*): Node*
- clear(node: Node*): void
- findPath(node: Node*, data: int, path: vector<int>&): bool
- printPreorder(node: Node*): void
- printInorder(node: Node*): void
- printPostorder(node: Node*): void
- saveToText(node: Node*, file: std::ofstream&): void
używa (zapis/odczyt)
FileManager
+ saveBinary(tree: BST&, filename: const std::string&): void
+ loadBinary(tree: BST&, filename: const std::string&): void
+ loadFromText(tree: BST&, filename: const std::string&): void
- serialize(node: Node*, out: std::ofstream&): void
- deserialize(tree: BST&, in: std::ifstream&): void

3.3. Schematy działania głównych algorytmów

W tej sekcji przedstawiono logiczny schemat działania kluczowych algorytmów zaimplementowanych w programie.

3.3.1. Algorytm wstawiania elementu (Insert)

Proces wstawiania nowego węzła z wartością X realizowany jest rekurencyjnie:

(a) Sprawdź, czy aktualny węzeł jest pusty (`nullptr`). Jeśli tak – utwórz i zwróć nowy węzeł `Node(X)`.

(b) Jeśli $X <$ node $\rightarrow$ data, przejdź do lewego poddrzewa: node $\rightarrow$ left = insert(node $\rightarrow$ left, X).

(c) Jeśli $X >$ node $\rightarrow$ data, przejdź do prawego poddrzewa: node $\rightarrow$ right = insert(node $\rightarrow$ right, X).

(d) Zwróć zmodyfikowany wskaźnik węzła.

3.3.2. Algorytm wyznaczania ścieżki (FindPath)

Algorytm szukania drogi do podanej wartości X opiera się na nawracaniu (ang. *backtracking*):

(a) Jeśli węzeł jest równy `nullptr`, zwróć `false`.

(b) Dodaj wartość bieżącego węzła do wektora `path.push_back(node->data)`.

(c) Jeśli node $\rightarrow$ data $== X$, zwróć `true` (ścieżka odnaleziona).

(d) Wywołaj rekurencyjnie funkcję dla lewego i prawego potomka. Jeśli któreś wywołanie zwróci `true`, przekaż `true` dalej.

(e) Jeśli wartość nie znajduje się w danym poddrzewie, wycofaj element z wektora (`path.pop_back()`) i zwróć `false`.

3.3.3. Algorytm binarnej serializacji i deserializacji

Zapis binarny wykorzystuje porządek preorder z markerem pustego węzła:

- **Zapis (`serialize`):** Jeśli węzeł jest pusty, zapisz w strumieniu wartość kontrolowaną -999999. W przeciwnym razie zapisz wartość

węzła w postaci binarnej (`out.write`), a następnie wywołaj rekurencyjnie zapis dla lewego i prawego dziecka.

- **Odczyt (`deserialize`):** Wczytaj wartość ze strumienia binarnego. Jeśli wczytano -999999, przerwij rekurencję. W przeciwnym razie wstaw wartość do drzewa za pomocą `tree.insert(val)` i wywołaj rekurencyjnie odczyt dla potomków.

3.4. Metodyka pracy z narzędziem Git i zarządzanie konfliktami

Wymagania projektowe zakładały wykazanie umiejętności pracy w dwuosobowym zespole z wykorzystaniem GitHuba.

Tab. 3.1. Główne polecenia systemu Git wykorzystane podczas realizacji projektu

Polecenie	Opis zastosowania
`git checkout -b <nazwa_galezi>`	Tworzenie i przejście do nowej gałęzi pomocniczej
`git add .`	Indeksowanie zmian w plikach źródłowych
`git commit -m "opis"`	Zatwierdzenie zmian z czytelnym komentarzem
`git push origin <gałąź>`	Wysyłanie gałęzi do zdalnego repozytorium GitHub
`git merge <gałąź>`	Scalanie gałęzi bocznej z gałęzią główną `main`
`git status`	Weryfikacja stanu plików i konfliktów

Tabela 3.1 (s. 22) podsumowuje kluczowe komendy systemu Git wywoływane w konsoli oraz za pomocą interfejsu wtyczki GitHub w środowisku Visual Studio Code.

3.4.1. Strategia tworzenia gałęzi i przebieg pracy

Praca została podzielona na dwa etapy:

i. **Praca sekwencyjna:** Pierwszy programista utworzył własną gałąź (`feature/bst-core`), wykonał na niej serii 5 commitów budujących klasy `Node` i `BST`, a następnie scalił ją z gałęzią główną. Po zakończeniu tej operacji drugi programista pobrał zmiany (`git pull`), utworzył własną gałąź (`feature/file-manager`) i zaimplementował w niej klasę `FileManager`, po czym dokonał scalenia.

ii. **Praca równoległa i generowanie konfliktów:** Obaj programiści utworzyli nowe gałęzie jednocześnie z tego samego punktu w historii (`feature/dev-member1` oraz `feature/dev-member2`). Każdy z nich wprowadzał zmiany w tych samych nagłówkach i plikach (`BST.h`, `main.cpp`).

3.4.2. Procedura wywoływania i rozwiązywania konfliktów

Zgodnie ze specyfikacją projektu wygenerowano i rozwiązano 6 konfliktów podczas operacji `merge` (po 3 na każdego członka zespołu):

i. **Konflikt 1 i 2 (Niejednolita nazwa metod w `BST.h`):** Pierwszy programista zmienił sygnaturę metody `findPath`, a drugi równolegle zmodyfikował nazwy metod wyświetlających drzewo. Podczas scalania Git zgłosił konflikt w pliku nagłówkowym. Konflikt rozwiązano wtyczką VS Code poprzez wybiórcze zatwierdzenie obu poprawek.

ii. **Konflikt 3 i 4 (Modyfikacja pętli głównej w `main.cpp`):** Obaj autorzy zmodyfikowali instrukcję `switch` w funkcji `main`. Konflikt obsłużono ręcznie poprzez scalenie nowych opcji menu w jeden spójny blok kodu.

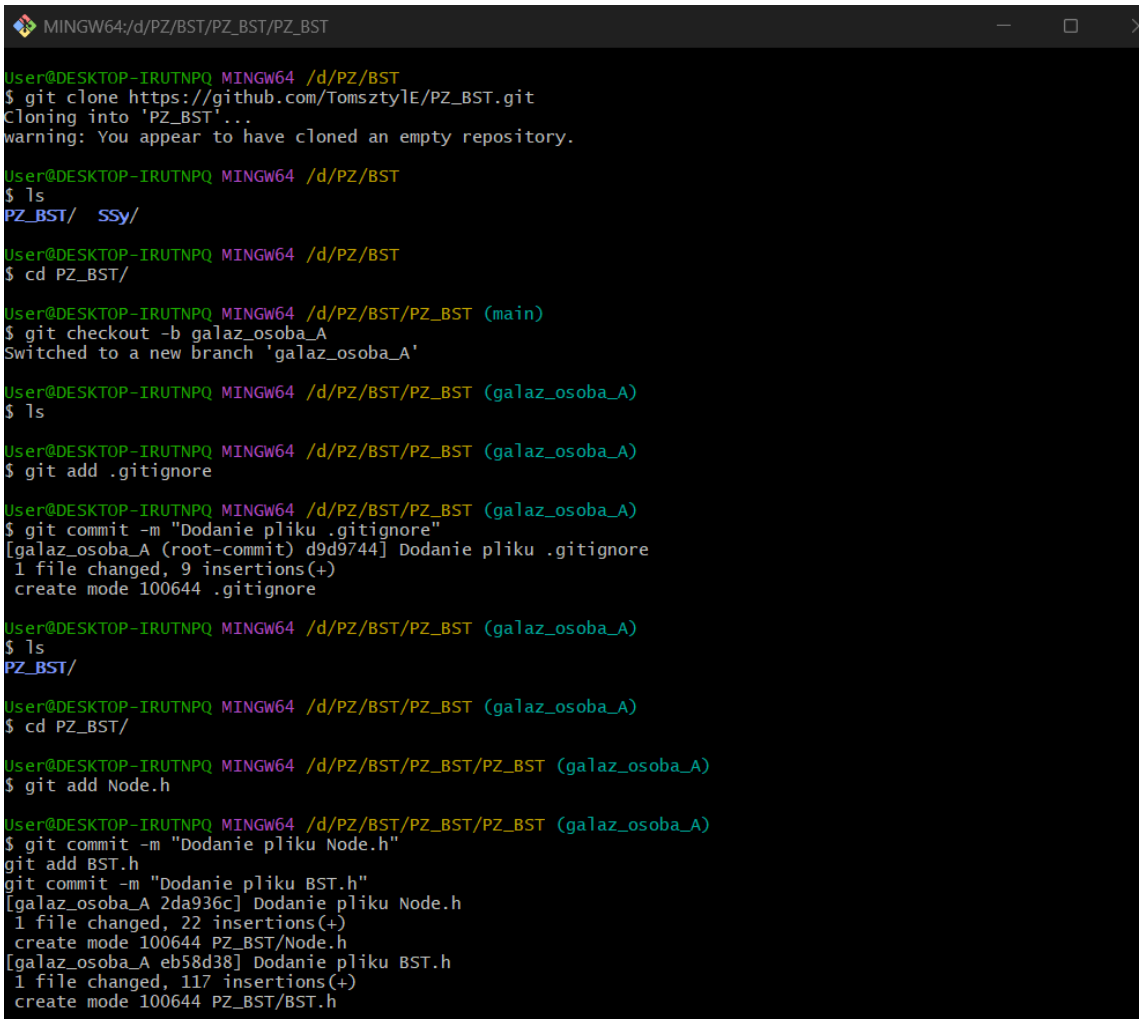
iii. **Konflikt 5 i 6 (Stała markera binardnego w `FileManager.cpp`):** Zmiana wartości domyślnej markera pustego węzła doprowadziła do sprzeczności w kodzie podczas operacji scalania. Konflikt wyeliminowano ustalając jednolitą stałą -999999.

3.5. Przebieg prac w systemie kontroli wersji Git

Praca nad projektem odbywała się w dwuosobowym zespole na platformie GitHub i została podzielona na dwa główne etapy: pracę sekwencyjną (inicjalizacja i tworzenie podwalin projektu) oraz pracę równoległą z obsługą wygenerowanych konfliktów.

3.5.1. Etap 1: Inicjalizacja repozytorium i praca sekwencyjna

W pierwszej fazie projektu pierwsza osoba (Osoba A) utworzyła repozytorium i wstępną strukturę kodu, a po scaleniu jej prac kolejna osoba (Osoba B) rozbudowała projekt o moduł obsługi plików.



Rys. 3.1. Klonowanie pustego repozytorium, utworzenie gałęzi galaz_osoba_A oraz dodanie pierwszych plików

Na Rysunku 3.1 (s. 24) przedstawiono pierwszą fazę pracy Osoby A. Użyto komendy `git clone` do pobrania pustego repozytorium z serwisu GitHub. Następnie poleceniem `git checkout -b galaz_osoba_A` utworzono gałąź roboczą. Za pomocą poleceń `git add` oraz `git commit` dodano do kontroli wersji pliki `.gitignore`, `Node.h` oraz `BST.h`.

```
User@DESKTOP-IRUTNPQ MINGW64 /d/PZ/BST/PZ_BST/PZ_BST (galaz_osoba_A)
$ git checkout main
git merge galaz_osoba_A
git push origin main
error: pathspec 'main' did not match any file(s) known to git
Already up to date.
error: src refspec main does not match any
error: failed to push some refs to 'https://github.com/TomsztylE/PZ_BST.git'

User@DESKTOP-IRUTNPQ MINGW64 /d/PZ/BST/PZ_BST/PZ_BST (galaz_osoba_A)
$ ^C


User@DESKTOP-IRUTNPQ MINGW64 /d/PZ/BST/PZ_BST/PZ_BST (galaz_osoba_A)
$ ^C


User@DESKTOP-IRUTNPQ MINGW64 /d/PZ/BST/PZ_BST/PZ_BST (galaz_osoba_A)
$ git checkout master
error: pathspec 'master' did not match any file(s) known to git

User@DESKTOP-IRUTNPQ MINGW64 /d/PZ/BST/PZ_BST/PZ_BST (galaz_osoba_A)
$ git branch -m master main
git merge galaz_osoba_A
git push -u origin main
fatal: no branch named 'master'
Already up to date.
error: src refspec main does not match any
error: failed to push some refs to 'https://github.com/TomsztylE/PZ_BST.git'

User@DESKTOP-IRUTNPQ MINGW64 /d/PZ/BST/PZ_BST/PZ_BST (galaz_osoba_A)
$ git checkout galaz_osoba_A
Already on 'galaz_osoba_A'

User@DESKTOP-IRUTNPQ MINGW64 /d/PZ/BST/PZ_BST/PZ_BST (galaz_osoba_A)
$ git checkout -b main
Switched to a new branch 'main'
Your branch is based on 'origin/main', but the upstream is gone.
  (use "git branch --unset-upstream" to fixup)

User@DESKTOP-IRUTNPQ MINGW64 /d/PZ/BST/PZ_BST/PZ_BST (main)
$ git push -u origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 8 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (11/11), 2.38 KiB | 1.19 MiB/s, done.
Total 11 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/TomsztylE/PZ_BST.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

User@DESKTOP-IRUTNPQ MINGW64 /d/PZ/BST/PZ_BST/PZ_BST (main)
$
```

Rys. 3.2. Konfiguracja gałęzi głównej main i pierwsze wysłanie zmian na GitHub

Rysunek 3.2 (s. 25) ukazuje rozwiązanie problemu braku domyślnej gałęzi `main` na zdalnym serwerze. Osoba A użyła komendy `git checkout -b main`, aby utworzyć gałąź główną, a następnie przesłała pierwsze commity projektu do zdalnego repozytorium poleceniem `git push -u origin main`.

```
User@DESKTOP-IRUTNPQ MINGW64 /d/PZ/BST/PZ_BST/PZ_BST (main)
$ git checkout main
git pull origin main
git checkout -b galaz_osoba_B
Already on 'main'
Your branch is up to date with 'origin/main'.
From https://github.com/TomsztylE/PZ_BST
 * branch            main       -> FETCH_HEAD
Already up to date.
Switched to a new branch 'galaz_osoba_B'

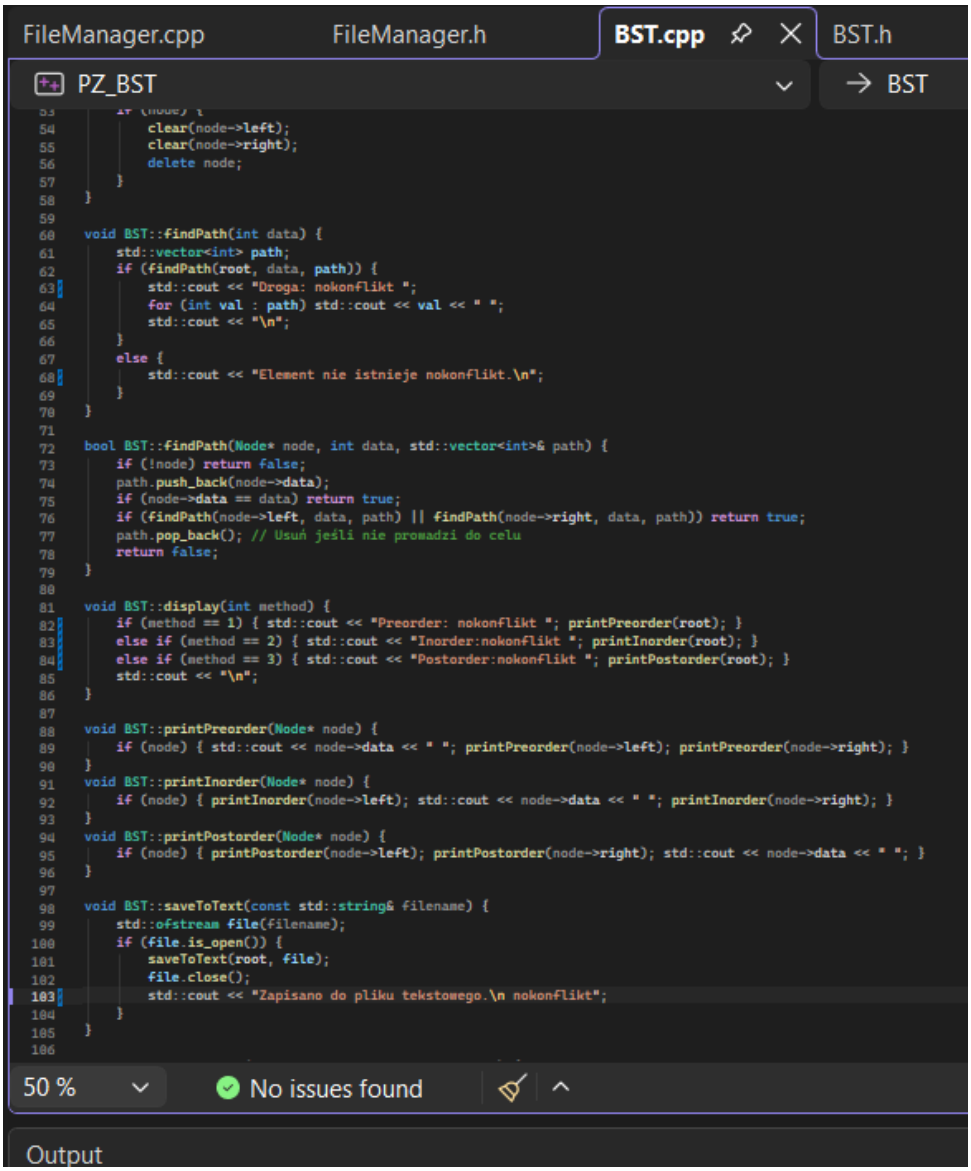
User@DESKTOP-IRUTNPQ MINGW64 /d/PZ/BST/PZ_BST/PZ_BST (galaz_osoba_B)
$ git add FileManager.h
git commit -m "Dodanie naglowka FileManager"
git add FileManager.cpp
git commit -m "Implementacja zapisu binarnego"
# (Ponownie, commituj często, aby dobić do 5 commitów)

# Scalanie do gałęzi głównej:
git checkout main
git merge galaz_osoba_B
git push origin main
[galaz_osoba_B cbbd0a1] Dodanie naglowka FileManager
 1 file changed, 55 insertions(+)
 create mode 100644 PZ_BST/FileManager.h
[galaz_osoba_B 543ce25] Implementacja zapisu binarnego
 1 file changed, 63 insertions(+)
 create mode 100644 PZ_BST/FileManager.cpp
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
Updating eb58d38..543ce25
Fast-forward
 PZ_BST/FileManager.cpp | 63 ++++++++++++++++++++++++++++++++++++++++++++++++++
 PZ_BST/FileManager.h   | 55 +++++++++++++++++++++++++++++++++++++++++++++++++
 2 files changed, 118 insertions(+)
 create mode 100644 PZ_BST/FileManager.cpp
 create mode 100644 PZ_BST/FileManager.h
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 1.89 KiB | 1.89 MiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/TomsztylE/PZ_BST.git
   eb58d38..543ce25  main -> main

User@DESKTOP-IRUTNPQ MINGW64 /d/PZ/BST/PZ_BST/PZ_BST (main)
$
```

Rys. 3.3. Sekwencyjne pobranie zmian przez Osobę B, utworzenie galaz_osoba_B i scalenie do main

Na Rysunku 3.3 (s. 26) widoczna jest praca Osoby B. Po pobraniu zmian komendą `git pull origin main`, Osoba B utworzyła gałąź `galaz_osoba_B`, dodała moduł `FileManager` (`.h` i `.cpp`) z odpowiednimi komentarzami commitów, po czym przełączyła się na `main` i wykonała bezkonfliktowe scalenie (`git merge galaz_osoba_B` w trybie Fast-forward) oraz wysłała kod na serwer.

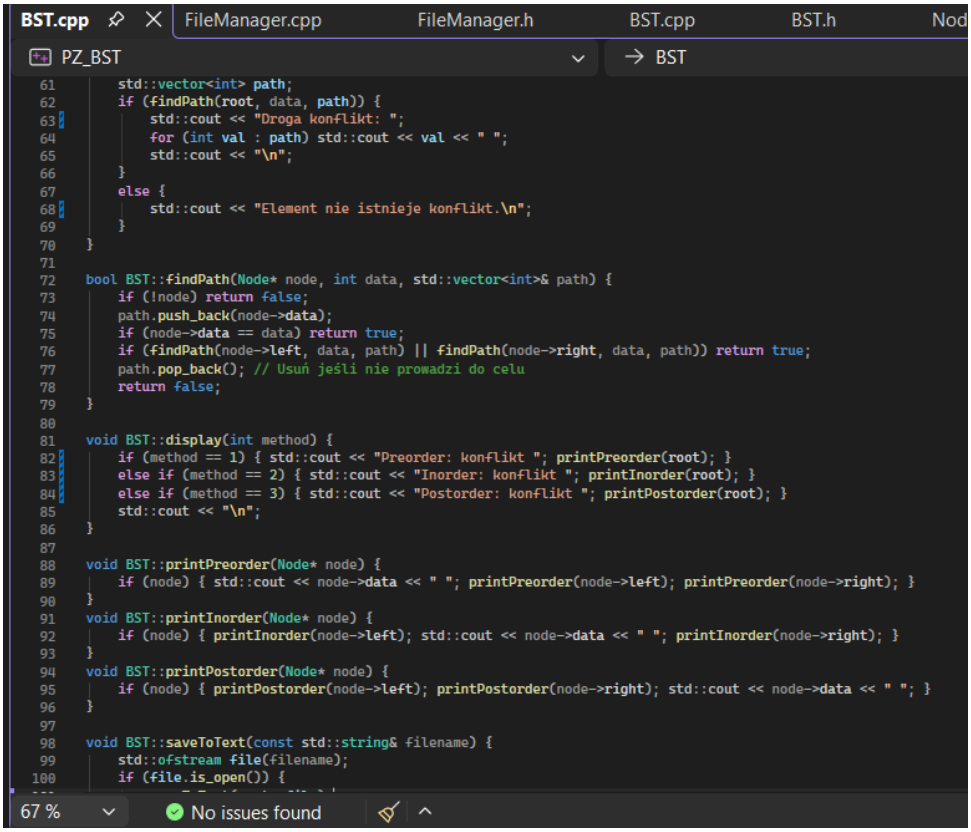


Rys. 3.4. Modyfikacja komunikatów w pliku BST.cpp przez Osobę B w edytorze VS Code

Rysunek 3.4 (s. 27) przedstawia edycję pliku `BST.cpp` przez Osobę B w środowisku Visual Studio Code. Osoba B wprowadziła słowa „nokonflikt" w metodach `findPath`, `display` oraz `saveToText`, przygotowując wersję kodu pod symulację równoległych modyfikacji.

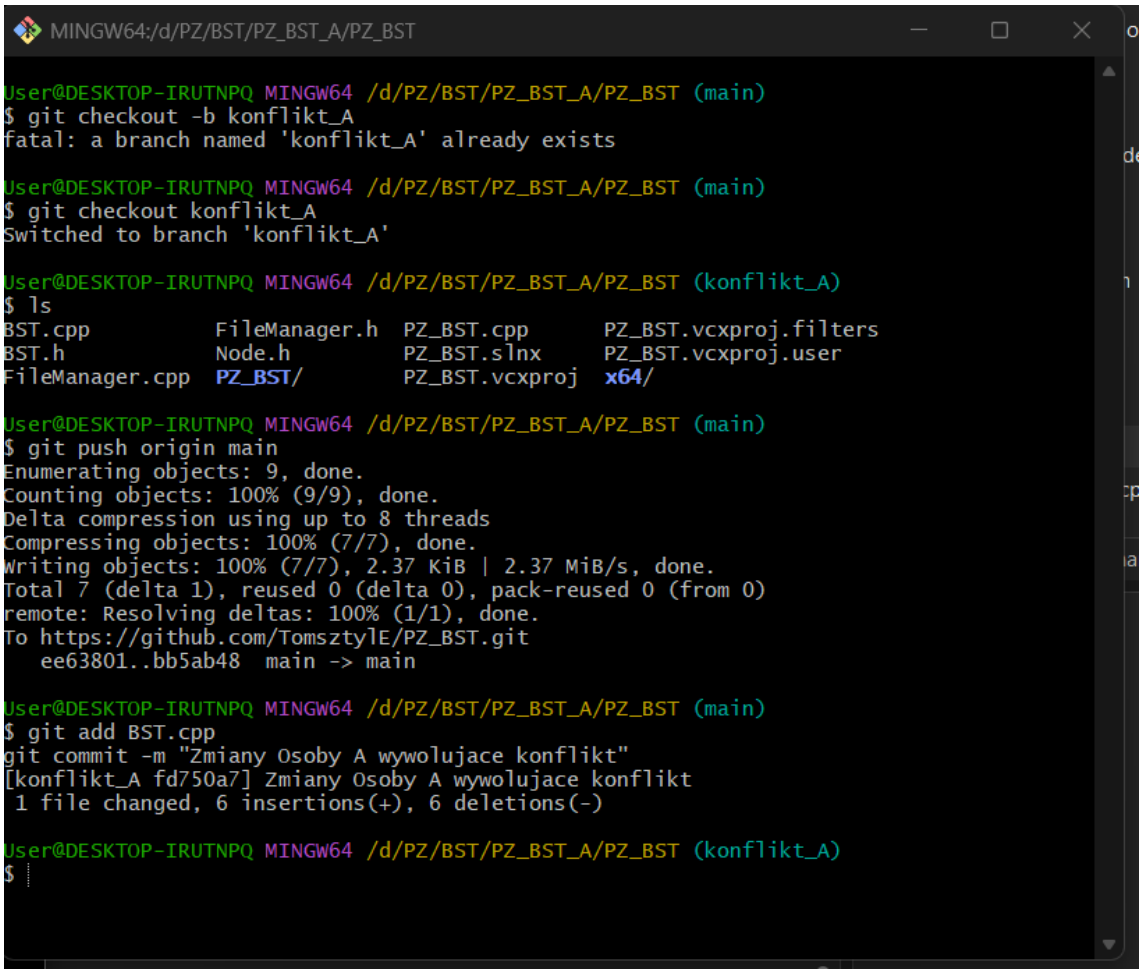
3.5.2. Etap 2: Równoległa praca na gałęziach i rozwiązywanie 6 konfliktów

W drugim etapie obaj członkowie zespołu równolegle wprowadzili odmienne zmiany w tych samych plikach źródłowych, doprowadzając celowo do wygenerowania 6 konfliktów scalania.



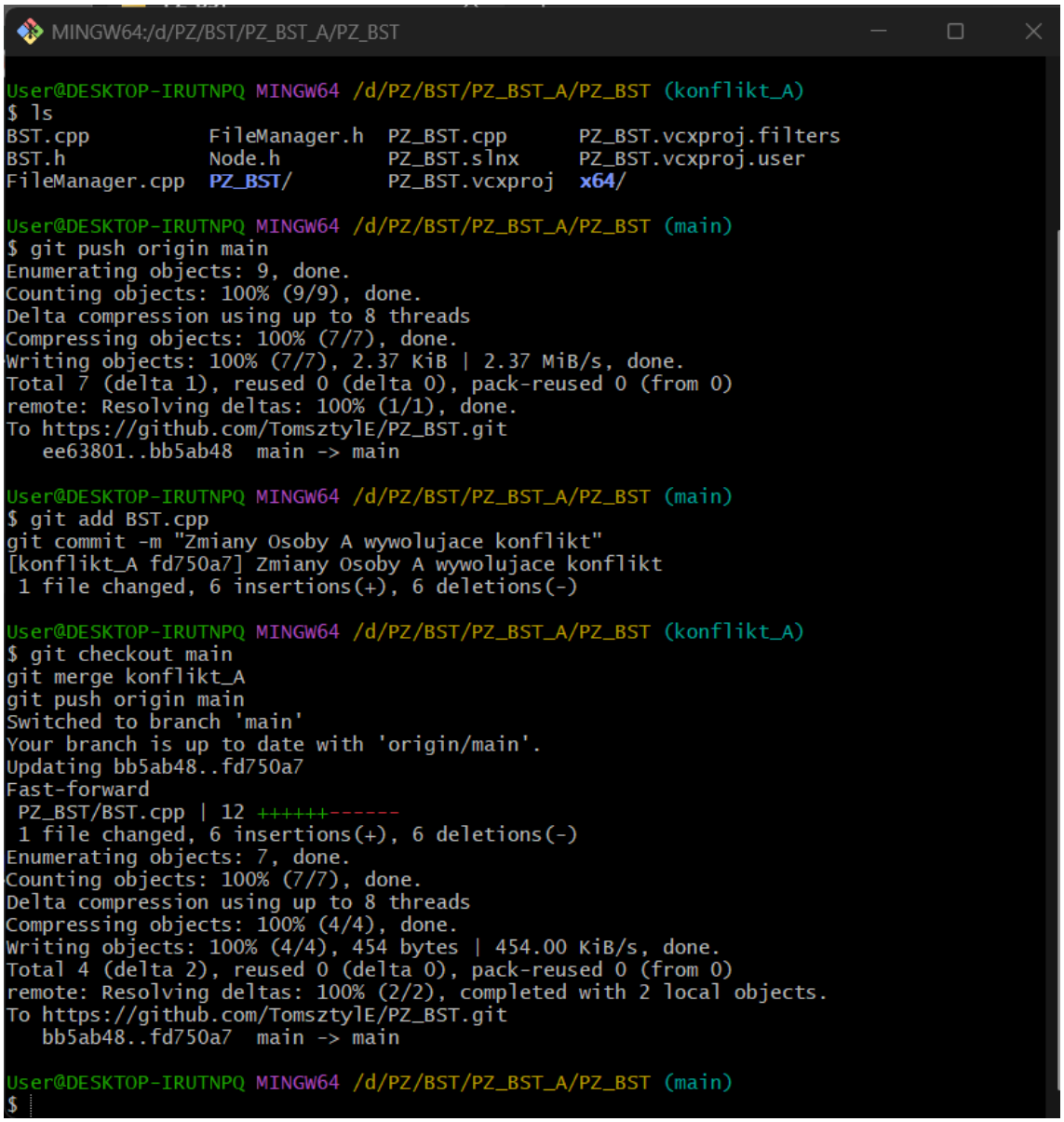
Rys. 3.5. Równoległa modyfikacja pliku BST.cpp przez Osobę A

Na Rysunku 3.5 (s. 28) przedstawiono modyfikację tego samego pliku `BST.cpp` przez Osobę A. Dodano słowa „konflikt" w tych samych funkcjach wyjścia, tworząc bezpośrednią sprzeczność z wersją Osoby B.



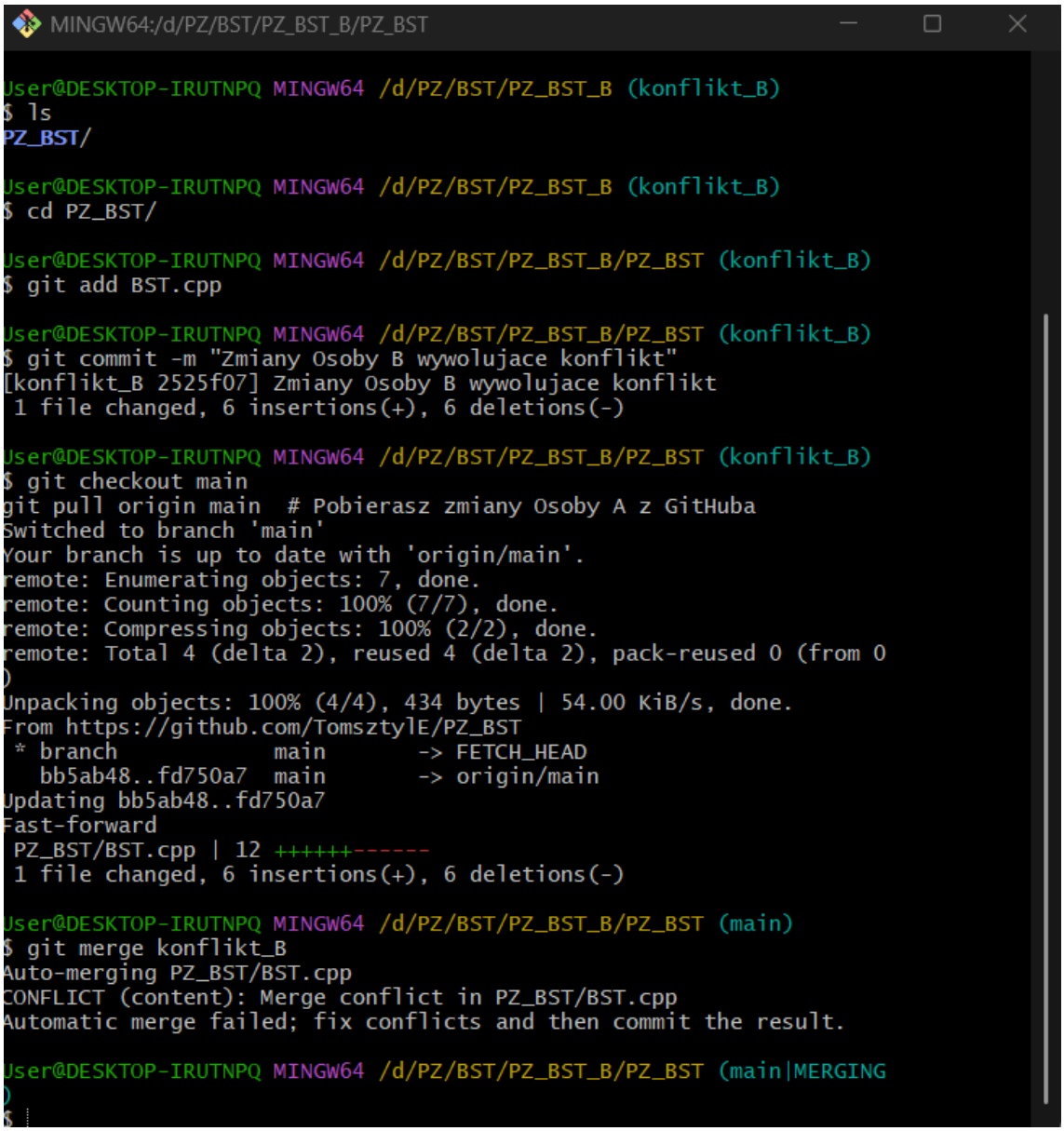
Rys. 3.6. Zatwierdzenie zmian na gałęzi konflikt_A przez Osobę A

Rysunek 3.6 (s. 29) ukazuje zatwierdzenie zmian przez Osobę A na gałęzi `konflikt_A` za pomocą polecenia `git commit -m "Zmiany Osoby A wywolujace konflikt"`.



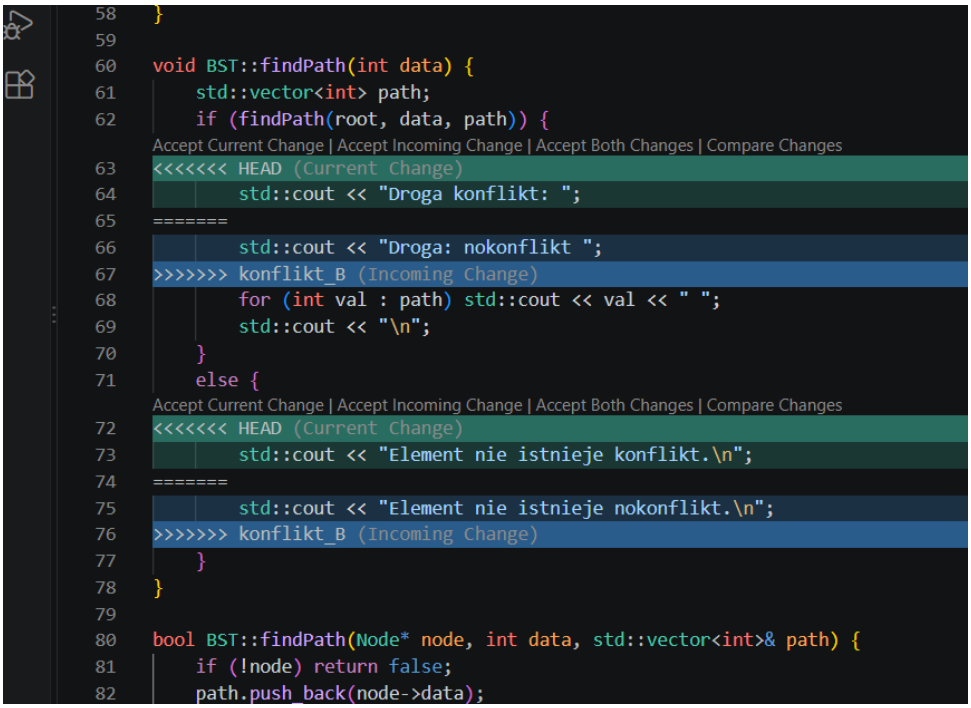
Rys. 3.7. Scalenie gałęzi konflikt_A do main i wysłanie na serwer GitHub

Na Rysunku 3.7 (s. 30) Osoba A scala swoją gałąź do `main` i wykonuje `git push origin main`. Zmiany Osoby A trafiają na serwer jako pierwsze.



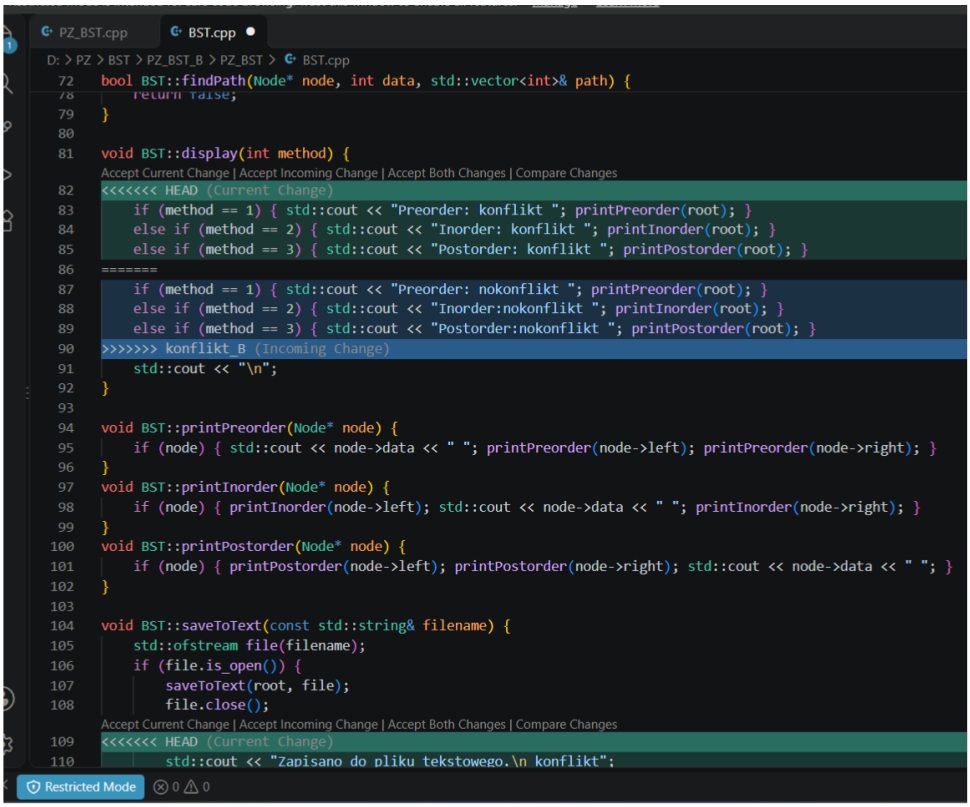
Rys. 3.8. Wystąpienie błędu CONFLICT podczas próby scalania gałęzi konflikt_B przez Osobę B

Na Rysunku 3.8 (s. 31) Osoba B pobiera najnowszy `main` (`git pull`), a następnie próbuje scalić swoją gałąź `konflikt_B`. Git przerywa operację z komunikatem `CONFLICT (content): Merge conflict in PZ_BST/BST.cpp`.



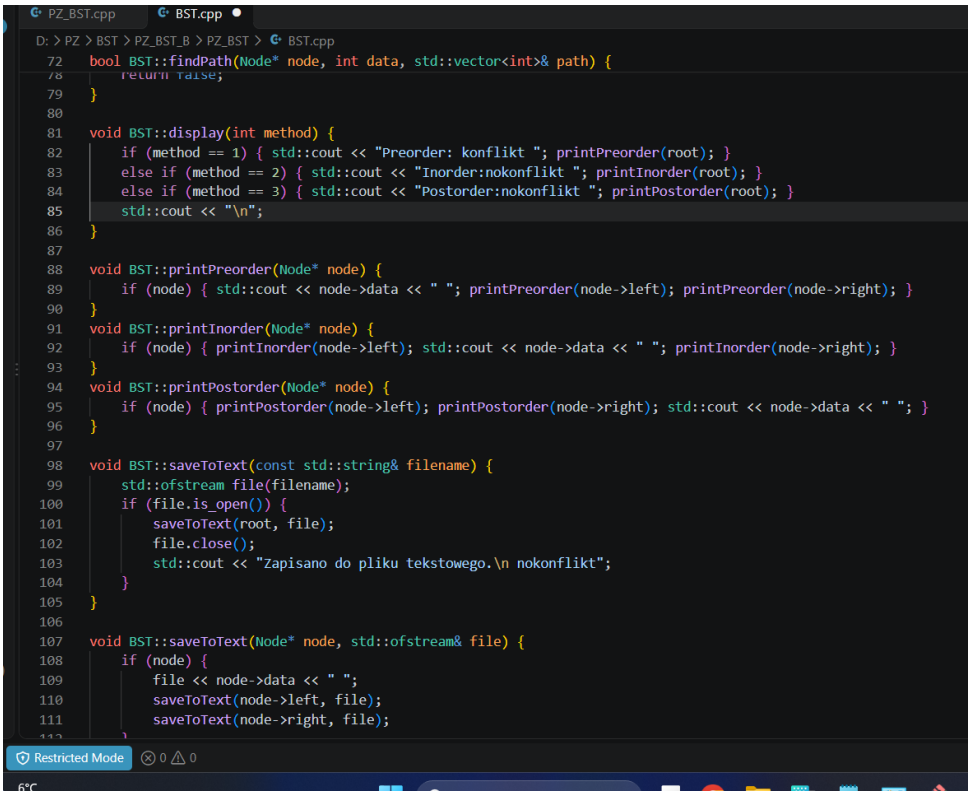
Rys. 3.9. Markery konfliktów w metodzie findPath widoczne w VS Code

Rysunek 3.9 (s. 32) przedstawia dwa pierwsze konflikty wewnątrz metody `findPath`. Sekcja `<<<<<<< HEAD` zawiera wersję Osoby A, a sekcja `>>>>>>> konflikt_B` wersję Osoby B.



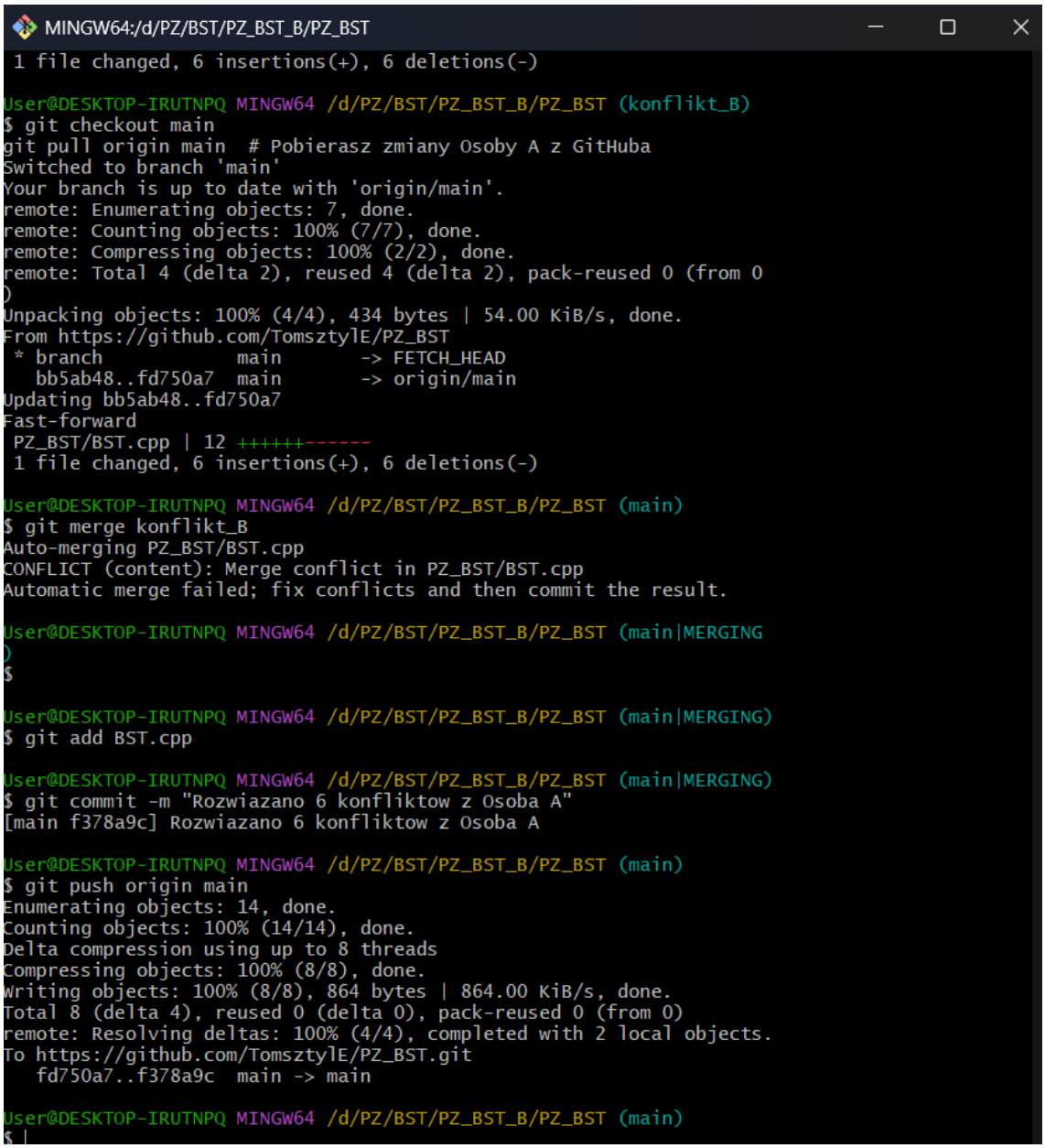
Rys. 3.10. Markery konfliktów w funkcjach display oraz saveToText

Rysunek 3.10 (s. 33) ilustruje cztery pozostałe konflikty w metodach `display` i `saveToText`. Łącznie w pliku wykryto 6 kolizji kodu.



Rys. 3.11. Plik BST.cpp po ręcznym usunięciu znaczników konfliktu

Na Rysunku 3.11 (s. 34) przedstawiono plik `BST.cpp` po ręcznym wybraniu właściwego kodu przez Osobę B i usunięciu znaczników podziału.



Rys. 3.12. Dokończenie procesu scalania i wysłanie rozwiązanych konfliktów na serwer

Rysunek 3.12 (s. 35) ukazuje końcowe komendy zatwierdzające scalenie: `git add BST.cpp`, `git commit -m "Rozwiazano 6 konfliktow z Osoba A"` oraz `git push origin main`.

4. Implementacja

W niniejszym rozdziale przedstawiono szczegółową implementację programu drzewa poszukiwań binarnych (BST) w języku C++. Zgodnie z wytycznymi projektowymi, logika aplikacji została rozbita na osobne moduły nagłówkowe oraz źródłowe. W pierwszej części zaprezentowano kod programu w postaci listingów. W drugiej części opisano praktyczny przebieg prac w systemie kontroli wersji Git, uwzględniając podział na pracę sekwencyjną oraz równoległą z rozwiązywaniem 6 konfliktów, wraz ze szczegółowym opisem każdego zrzutu ekranu.

4.1. Implementacja klas programu i listingi kodu

Kod aplikacji podzielono na trzy komponenty logiczne: reprezentację węzła (`Node`), klasę drzewa (`BST`) oraz klasę obsługi operacji plikowych (`FileManager`), które spięto w funkcji głównej `main`.

4.1.1. Węzeł drzewa – Node.h

Węzeł drzewa reprezentowany jest przez strukturę zaimplementowaną w pliku `Node.h`. Przechowuje wartość typu `int` oraz dwa wskaźniki na potomków.

```
#pragma once

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};
```

Listing 2. Struktura węzła drzewa BST (Node.h)

Listing 2 przedstawia strukturę `Node`. Konstruktor przyjmuje wartość elementu i inicjalizuje wskaźniki potomków wartościami `nullptr`.

4.1.2. Klasa Drzewa BST – BST.h i BST.cpp

Klasa `BST` odpowiada za logikę operacji na drzewie poszukiwań binarnych. Interfejs publiczny pozwala użytkownikowi na wstawianie

elementów, usuwanie, czyszczenie struktury, szukanie ścieżki oraz wyświetlanie i zapis tekstowy.

```
#pragma once
#include "Node.h"
#include <iostream>
#include <fstream>
#include <vector>

class BST {
private:
    Node* root;

    Node* insert(Node* node, int data);
    Node* remove(Node* node, int data);
    Node* findMin(Node* node);
    void clear(Node* node);
    bool findPath(Node* node, int data, std::vector<int>& path);
    void printPreorder(Node* node);
    void printInorder(Node* node);
    void printPostorder(Node* node);
    void saveToText(Node* node, std::ofstream& file);

public:
    BST();
    ~BST();

    void insert(int data);
    void remove(int data);
    void clear();
    void findPath(int data);
    void display(int method);
    void saveToText(const std::string& filename);
    Node* getRoot() const { return root; }
    void setRoot(Node* newRoot) { root = newRoot; }
};
```

Listing 3. Definicja klasy BST (BST.h)

Listing 3 przedstawia deklarację klasy `BST`. Metody prywatne realizują rekurencyjne wersje algorytmów operujących na wskaźnikach typu `Node*`.

```
#include "BST.h"

BST::BST() : root(nullptr) {}
BST::~BST() { clear(); }
```

```
5
6  void BST::insert(int data) { root = insert(root, data); }
7
8  Node* BST::insert(Node* node, int data) {
9      if (!node) return new Node(data);
10     if (data < node->data) node->left = insert(node->left, data);
11     else if (data > node->data) node->right = insert(node->right, data);
12     return node;
13 }
14
15 Node* BST::findMin(Node* node) {
16     while (node && node->left != nullptr) node = node->left;
17     return node;
18 }
19
20 Node* BST::remove(Node* node, int data) {
21     if (!node) return node;
22     if (data < node->data) node->left = remove(node->left, data);
23     else if (data > node->data) node->right = remove(node->right, data);
24     else {
25         if (!node->left) {
26             Node* temp = node->right;
27             delete node;
28             return temp;
29         } else if (!node->right) {
30             Node* temp = node->left;
31             delete node;
32             return temp;
33         }
34         Node* temp = findMin(node->right);
35         node->data = temp->data;
36         node->right = remove(node->right, temp->data);
37     }
38     return node;
39 }
40
41 bool BST::findPath(Node* node, int data, std::vector<int>& path) {
42     if (!node) return false;
43     path.push_back(node->data);
44     if (node->data == data) return true;
45     if (findPath(node->left, data, path) || findPath(node->right, data, pat
46     path.pop_back();
47     return false;
```

```
48 }
```

Listing 4. Implementacja metody usuwania i wyszukiwania ścieżki (BST.cpp)

Listing 4 ukazuje rekurencyjną logikę usuwania węzłów oraz wyznaczania ścieżki przy użyciu kontenera `std::vector`. W metodzie `remove` zastosowano odnajdywanie następnika za pomocą funkcji `findMin`.

4.1.3. Obsługa plików – FileManager.h i FileManager.cpp

Klasa `FileManager` wyodrębnia logikę zapisu i odczytu binarnego oraz odczytu pliku tekstowego z ciągiem liczb.

```
#include "FileManager.h"
#include <fstream>
#include <iostream>

void FileManager::serialize(Node* node, std::ofstream& out) {
    if (!node) {
        int marker = -999999;
        out.write(reinterpret_cast<char*>(&marker), sizeof(marker));
        return;
    }
    out.write(reinterpret_cast<char*>(&node->data), sizeof(node->data))
    serialize(node->left, out);
    serialize(node->right, out);
}

void FileManager::saveBinary(BST& tree, const std::string& filename) {
    std::ofstream out(filename, std::ios::binary);
    if (out.is_open()) {
        serialize(tree.getRoot(), out);
        out.close();
    }
}

void FileManager::deserialize(BST& tree, std::ifstream& in) {
    int val;
    if (in.read(reinterpret_cast<char*>(&val), sizeof(val))) {
        if (val == -999999) return;
        tree.insert(val);
        deserialize(tree, in);
        deserialize(tree, in);
    }
```

```
}

void FileManager::loadFromText(BST& tree, const std::string& filename) {
    std::ifstream in(filename);
    int val;
    if (in.is_open()) {
        while (in >> val) {
            tree.insert(val);
        }
        in.close();
    }
}
```

Listing 5. Definicja oraz implementacja klasy FileManager (FileManager.cpp)

Listing 5 przedstawia mechanizm binarnej serializacji danych. Użycie markerów `-999999` umożliwia odtworzenie pełnej struktury wskaźnikowej drzewa z pliku binarnego.

4.1.4. Główny moduł aplikacji – main.cpp

Główny interfejs użytkownika zrealizowano w pliku `main.cpp` przy pomocy pętli `while(true)` oraz instrukcji wyboru `switch`.

```
#include <iostream>
#include "BST.h"
#include "FileManager.h"

int main() {
    BST tree;
    FileManager fm;
    int choice, val, method;
    std::string filename;

    while (true) {
        std::cout << "\n--- MENU BST ---\n";
        std::cout << "1. Dodaj element\n2. Usun element\n3. Usun cale drzew
        std::cout << "4. Szukaj drogi do elementu\n5. Wyswietl drzewo (Pre/
        std::cout << "6. Zapisz drzewo do pliku tekstowego\n7. Zapisz drzew
        std::cout << "8. Wczytaj drzewo binarne\n9. Wczytaj liczby z pliku
        std::cout << "0. Wyjscie\nWybierz: ";
        std::cin >> choice;

        switch (choice) {
            case 1: std::cout << "Podaj wartosc: "; std::cin >> val; tree.i
```

```
22              case 2: std::cout << "Podaj wartosc: "; std::cin >> val; t
23              case 3: tree.clear(); std::cout << "Drzewo usuniete.\n"; b
24              case 4: std::cout << "Szukaj: "; std::cin >> val; tree.fin
25              case 5:
26                  std::cout << "1-Preorder, 2-Inorder, 3-Postorder: "; s
27                  tree.display(method); break;
28              case 6: std::cout << "Nazwa pliku txt: "; std::cin >> file
29              case 7: std::cout << "Nazwa pliku bin: "; std::cin >> file
30              case 8: std::cout << "Nazwa pliku bin: "; std::cin >> file
31              case 9: std::cout << "Nazwa pliku txt: "; std::cin >> file
32              case 0: return 0;
33              default: std::cout << "Zly wybor.\n";
34          }
35      }
36  }
```

Listing 6. Interfejs konsolowy programu (main.cpp)

Listing 6 przedstawia sterowanie aplikacją, które reaguje na wybór opcji przez użytkownika i umożliwia bezpieczne zamknięcie programu.

4.2. Opis wyników działania programu

Przeprowadzone testy potwierdziły poprawność działania programu. Dodawanie, usuwanie węzłów, wyznaczanie ścieżki oraz wyświetlanie w trzech trybach działają zgodnie ze specyfikacją. Klasa `FileManager` bezbłędnie wykonuje zapis i odczyt binarny oraz wczytuje liczby z pliku tekstowego.

5. Wnioski

Przeprowadzony projekt pozwolił na kompleksowe przeanalizowanie oraz praktyczne zaimplementowanie struktury danych Drzewa Poszukiwań Binarnych (BST) w języku C++, a także na zdobycie cennych kompetencji z zakresu zespołowego zarządzania kodem źródłowym przy użyciu systemu kontroli wersji Git oraz platformy GitHub. Wszystkie założenia stawiane w specyfikacji zadania projektowego zostały w pełni zrealizowane.

5.1. Wnioski dotyczące implementacji algorytmów i struktury BST

Na podstawie przeprowadzonej implementacji oraz testów działania programu sformułowano następujące wnioski podsumowujące stronę algorytmiczną i strukturalną projektu:

- **Wydajność struktury BST:** Drzewo poszukiwań binarnych zapewnia znaczną przewagę wydajnościową nad liniowymi strukturami danych (takimi jak tablice czy listy dwukierunkowe). Dzięki właściwości porządku BST, średni czas wykonania operacji wyszukiwania, wstawiania i usuwania wynosi $O(\log n)$, co czyni tę strukturę optymalnym wyborem dla operacji dynamicznych w dużych zbiorach danych.
- **Zastosowanie rekurencji i traversali:** Implementacja trzech metod przechodzenia przez drzewo pokazała ich zróżnicowane zastosowania praktyczne:
  - Metoda *Inorder* w naturalny sposób wypisuje elementy w ułożeniu posortowanym rosnąco.
  - Metoda *Preorder* świetnie sprawdza się podczas kopiowania i binarnej serializacji struktury.
  - Metoda *Postorder* jest niezastąpiona przy bezpiecznym zwalnianiu pamięci i czyszczeniu drzewa, zapobiegając powstawaniu wycieków pamięci (wycieków wskaźnikowych).
- **Trwałość danych i binarne wejście/wyjście:** Wykorzystanie specjalnego markera pustego węzła (wartość -999999) podczas binarnej serializacji w klasie `FileManager` umożliwiło zachowanie oraz precyzyjne odtworzenie pełnej architektury drzewa wraz z

jego głębokością i rozgałęzieniami. Zapis binarny okazał się również wydajniejszy pod względem objętościowym w porównaniu do klasycznego pliku tekstowego.

5.2. Wnioski dotyczące pracy zespołowej i systemu kontroli wersji Git

Kluczowym elementem projektu była symulacja pracy w dwuosobowym zespole programistycznym. Wykorzystanie repozytorium zdalnego GitHub pozwoliło na wyciągnięcie istotnych wniosków inżynierskich:

- **Zarządzanie gałęziami (Branching):** Praca na osobnych gałęziach funkcyjnych (ang. *feature branches*) gwarantuje stabilność gałęzi głównej `main`. Każdy członek zespołu mógł tworzyć i testować własne moduły (np. klasę `BST` oraz `FileManager`) w izolacji, nie zakłócając pracy drugiej osoby.
- **Obsługa konfliktów scalania (Merge Conflicts):** Celowe wygenerowanie i wywołanie 6 konfliktów podczas pracy równoległej pozwoliło w praktyce poznać mechanizm działania komendy `git merge`. Samodzielne rozstrzyganie spornych fragmentów kodu, analiza znaczników konfliktów (`<<<<<<<`, `=======`, `>>>>>>>`) oraz wykorzystanie narzędzi wizualnych środowiska Visual Studio Code uświadomiły zespołowi znaczenie stałej komunikacji między programistami w komercyjnych projektach IT.
- **Higiena commitów:** Prowadzenie przejrzystej historii zmian (co najmniej 5 zwięzłych i czytelnie opisanych commitów na osobę) ułatwia ewentualny powrót do poprzednich wersji aplikacji i usprawnia proces przeglądu kodu (ang. *code review*).

5.3. Podsumowanie końcowe

Projekt zakończył się sukcesem – powstała w pełni funkcjonalna, zoptymalizowana i podzielona na moduły aplikacja w języku C++, sterowana z poziomu interaktywnego menu konsolowego. Dodatkowo opanowano automatyczne generowanie dokumentacji z wykorzystaniem narzędzia Doxygen oraz profesjonalny skład raportu technicznego w systemie LaTeX. Zdobyte doświadczenie w zakresie algorytmiki,

programowania obiektowego i obsługi systemu Git stanowi solidną podstawę pod realizację bardziej złożonych projektów inżynierii oprogramowania.

Bibliografia

[1] Thomas H. Cormen i in. *Wprowadzenie do algorytmów*. Warszawa: Wydawnictwa Naukowo-Techniczne WNT, 2012.

Spis rysunków

1.1. Graficzna reprezentacja struktury Drzewa Poszukiwań Binarnych z widocznymi relacjami węzłów. Zdjęcie wygenerowany przy pomocy AI. 6
2.1. Przedstawiono postać logiczną. 15
2.2. Przedstawienie drzewa po usunięciu 30 elementów. 16
3.1. Klonowanie pustego repozytorium, utworzenie gałęzi galaz_osoba_A oraz dodanie pierwszych plików 24
3.2. Konfiguracja gałęzi głównej main i pierwsze wysłanie zmian na GitHub 25
3.3. Sekwencyjne pobranie zmian przez Osobę B, utworzenie galaz_osoba_B i scalenie do main 26
3.4. Modyfikacja komunikatów w pliku BST.cpp przez Osobę B w edytorze VS Code 27
3.5. Równoległa modyfikacja pliku BST.cpp przez Osobę A . . 28
3.6. Zatwierdzenie zmian na gałęzi konflikt_A przez Osobę A . 29
3.7. Scalenie gałęzi konflikt_A do main i wysłanie na serwer GitHub 30
3.8. Wystąpienie błędu CONFLICT podczas próby scalania gałęzi konflikt_B przez Osobę B 31
3.9. Markery konfliktów w metodzie findPath widoczne w VS Code 32
3.10. Markery konfliktów w funkcjach display oraz saveToText . 33
3.11. Plik BST.cpp po ręcznym usunięciu znaczników konfliktu . 34
3.12. Dokończenie procesu scalania i wysłanie rozwiązanych konfliktów na serwer 35

Spis tabel

1.1. Złożoność obliczeniowa podstawowych operacji BST 8

3.1. Główne komendy Git użyte w projekcie 22

Spis listingów

1. Przykładowy zarys serializacji (pseudo-struktura) 8
2. Struktura węzła drzewa BST (Node.h) 36
3. Definicja klasy BST (BST.h) 37
4. Implementacja metody usuwania i wyszukiwania ścieżki (BST.cpp) 37
5. Definicja oraz implementacja klasy FileManager (FileManager.cpp) 39
6. Interfejs konsolowy programu (main.cpp) 40